PROXMOX / LXC / DOCKER / NGINX PROXY MANAGER

Project Documentation

Virtualisation and reverse-proxy deployment overview

Prepared for administration, troubleshooting and future maintenance of the Proxmox-based service environment.

Project environment: Proxmox VE 9.2.2 · Debian 12 LXC · Docker · Nginx Proxy Manager

1. What this project does

This project uses Proxmox VE to run a lightweight Linux container for Docker-based services. Nginx Proxy Manager (NPM) runs in the container and provides a web interface for managing reverse-proxy hosts and TLS certificates.

The reverse proxy accepts web requests for configured hostnames and forwards each request to the relevant application on the private network. This allows multiple web services to use separate domain names while sharing the same proxy service.

The documentation describes the existing Proxmox container and the intended NPM role. It does not assume that every sample Docker setting or proxy host shown here is currently active.

2. Current project details

Item	Current value
Virtualisation platform	Proxmox VE 9.2.2
Proxmox node	pve
Container ID	110
Container name	nginx-proxy
Container operating system	Debian GNU/Linux 12 (Bookworm)
Container address	192.168.50.3/24
Default gateway	192.168.50.1
Container CPU allocation	1 vCPU
Container memory	512 MiB RAM and 512 MiB swap
Container root disk	8 GB on local-lvm
Main service	Nginx Proxy Manager running through Docker

The values above were visible in the project setup information. Confirm the live container settings in Proxmox before making resource or network changes.

3. How a request reaches an application

A normal web request follows this general path:

```
Visitor's browser
      |
      | HTTPS request for a configured domain
      v
Public DNS / internet edge
      |
      | TCP 80 and 443 reach the NPM service
      v
Nginx Proxy Manager
      |  Selects a Proxy Host by domain name
      |  Terminates HTTPS and applies the certificate
      v
Target application on the private network
      |
```

```
        v
Application response returns through NPM
```

NPM acts as the entry point for the web services configured in it. Each Proxy Host contains a domain name and the destination address and port of its application. The target application does not need to be exposed directly to the internet when the proxy design is used.

4. Proxmox container

The NPM service is hosted in Proxmox container 110, named nginx-proxy. The container is running Debian 12 and has the private address 192.168.50.3/24 with gateway 192.168.50.1.

The container was able to reach an external IP address during the network check. This confirmed basic outbound connectivity at the time of testing; it does not by itself confirm that inbound web access or domain routing is working.

Useful checks from the Proxmox shell:

```
pct status 110
pct config 110
pct exec 110 -- ip addr
pct exec 110 -- ip route
```

Use the Proxmox interface or the pct commands to review the container before changing its network, storage or resource allocation. Avoid changing the container address without also updating any services that use it.

5. Docker and Nginx Proxy Manager

Docker provides the runtime for NPM inside the Debian container. The exact Compose file location, container names and volume paths should be checked on the running system rather than assumed from a generic installation example.

Useful checks inside the container:

```
docker ps
 docker compose ls
 docker volume ls
```

If NPM was deployed with Docker Compose, move to the directory containing the existing compose.yaml or docker-compose.yml file before using Compose commands. Do not create a second NPM stack until the current deployment and its persistent data locations have been identified.

A common NPM deployment uses the following ports:

Port	Purpose
80/tcp	HTTP traffic and HTTP-01 certificate validation
81/tcp	NPM administration interface
443/tcp	HTTPS traffic

The administration interface on port 81 should remain restricted to trusted management access. Do not expose it publicly unless there is a specific, secured management requirement.

6. Proxy Host and HTTPS settings

For each application, create a Proxy Host in the NPM dashboard. Enter the application's hostname, destination scheme, private IP address and listening port. Enable the required WebSocket or access options only when the application needs them.

NPM setting	Value / guidance
Domain Names	The public hostname assigned to the application
Scheme	HTTP or HTTPS, matching the target application
Forward Hostname / IP	Private IP address or resolvable service name
Forward Port	The port on which the target application listens
SSL certificate	Select an issued certificate for the hostname
Force SSL	Enable after HTTPS has been tested
HTTP/2 Support	Enable when appropriate for the service

A certificate request can fail if the hostname does not resolve to the correct public endpoint, the required validation path is unavailable, or the certificate authority cannot reach the challenge. Check the hostname and validation method before repeatedly requesting certificates.

The existing dashboard showed a certificate for pve.mahfuj.bd. A successful certificate for that hostname does not prove that other hostnames have valid DNS, proxy-host or certificate settings.

7. Security and maintenance

- Keep Proxmox, Debian and Docker updated using supported release and package channels.
- Keep the NPM administration interface private and use strong, unique administrator credentials.
- Expose only the web ports required for the services. Avoid direct public access to application backends when NPM is intended to be the entry point.
- Back up the NPM database and persistent data volumes, including certificate data, before upgrades or major changes.
- Record the location of the Docker Compose file and the persistent volume mappings after verifying them on the host.
- Check container resource usage periodically. The current allocation is 1 vCPU and 512 MiB RAM, so monitor memory pressure as additional proxy hosts or services are added.
- Test a restore procedure; a backup is useful only if the configuration and certificate data can be recovered.

8. Checks before handover

- Container 110 is running and its address and gateway are correct.
- The container can resolve DNS names and reach required network destinations.
- Docker is running and the intended NPM containers are healthy.
- The NPM dashboard is reachable from the trusted management network.
- Each required Proxy Host points to the correct private application address and port.
- The public hostname resolves to the intended endpoint.
- HTTPS opens without a browser certificate warning.
- The application works through its public hostname, including any required WebSocket functions.
- NPM data and certificate files are included in a tested backup.
- The service returns after a planned container or Proxmox restart.

9. Details still to be recorded

The available project information confirms the Proxmox version, node, container ID and name, Debian release, container IP address, gateway and allocated resources. The following details should be recorded after checking the live environment:

- Docker Compose file path and current service definitions.
- Actual Docker container names, image versions and health status.
- NPM persistent volume paths and database location.
- NPM Proxy Host entries and their backend destinations.
- NPM host's public access method and the current DNS / router configuration.
- Certificate names, expiry dates and renewal method.
- Backup destination, schedule and most recent successful restore test.

Keep this document updated whenever the container address, Docker deployment, proxy destinations or certificate configuration changes.

Result: https://pve.mahfuj.bd

(**Note:** if you don't find the site using this link, probably I have removed the server)

